

VIBE: DEVELOPMENT OF A SOCIAL PLATFORM FOR MICROBLOGGING

DAVID SANTIAGO TELLEZ MELO-JOHAN NICOLAS LOPEZ HERNANDEZ



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

INTRODUCTION

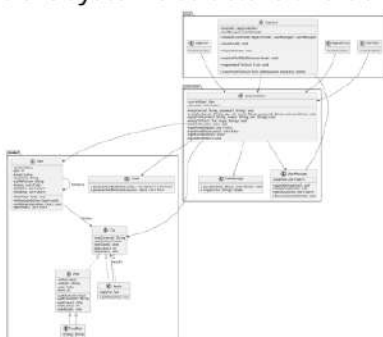
Social networks have become essential platforms for real-time communication, interaction, and content sharing. Inspired by microblogging services like Twitter, Vibe is a desktop application that allows users to post short messages (up to 280 characters), upload images or links, and interact with others through likes and replies. The project follows an object-oriented approach and is structured using the Model-View-Controller (MVC) architecture, ensuring clear separation of concerns and maintainability. Vibe aims to provide a lightweight yet functional social experience focused on simplicity, usability, and efficient user interaction.

GOAL

To develop a simple and functional microblogging application that enables users to share short posts and interact in an intuitive environment.

METHODS

We followed the object-oriented programming paradigm and the Model-View-Controller (MVC) architectural pattern to organize the system into logical components. The model layer defines the application's core entities such as users, posts, replies, and feeds, allowing encapsulation of behaviors like following users or liking content. The controller layer handles business logic, including data persistence, feed management, and interactions. The view layer includes multiple graphical interfaces such as login, registration, and the main screen, built using Java Swing. We used CRC cards to assign responsibilities to each class and identify their collaborators. UML class diagrams and user flow illustrations were also created to visualize the system's structure and behavior.



RESULTS

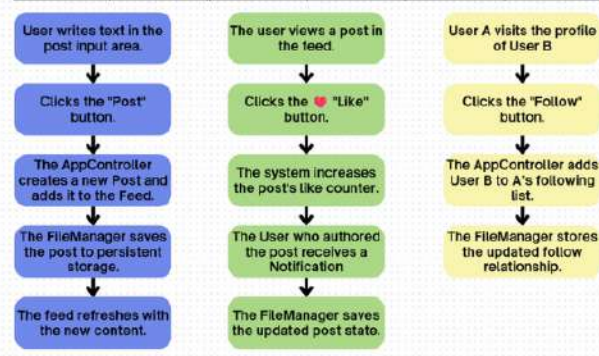
The development of Vibe resulted in a fully functional desktop-based microblogging application. The system supports key features such as user registration and login, posting short messages (up to 280 characters), uploading images and links, liking content, viewing other user profiles, and following/unfollowing users. The system was successfully implemented using Java and Swing, with object persistence managed through custom serialization. Each functional requirement was mapped to a corresponding component in the model, controller, or view layer, allowing for modular and maintainable development.

Class: Messenger	Class: User	Class: Post
Responsibilities: - Send and receive messages via the GUI. - Manage the state of the application.	Responsibilities: - Display the messages (posts, replies, etc.) in the GUI. - Manage the state of the application.	Responsibilities: - Display the messages (posts, replies, etc.) in the GUI. - Manage the state of the application.
Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply

Class: Reply	Class: UserManager	Class: LoginGUI
Responsibilities: - Display the replies to a post. - Manage the state of the application.	Responsibilities: - Manage the state of the application. - Manage the state of the application.	Responsibilities: - Manage the state of the application. - Manage the state of the application.
Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply

Class: Feed	Class: AppController	Class: RegisterGUI
Responsibilities: - Display the feed of posts. - Manage the state of the application.	Responsibilities: - Manage the state of the application. - Manage the state of the application.	Responsibilities: - Manage the state of the application. - Manage the state of the application.
Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply

Class: Feed	Class: RegisterGUI	Class: LoginGUI
Responsibilities: - Display the feed of posts. - Manage the state of the application.	Responsibilities: - Manage the state of the application. - Manage the state of the application.	Responsibilities: - Manage the state of the application. - Manage the state of the application.
Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply	Collaborators: - AppController - FileManager - Feed - Reply



FUTURE DIRECTIONS

To extend the capabilities of Vibe, future work will focus on implementing a cloud-based database for persistent data storage, real-time notifications using socket programming, and a web or mobile version of the application for broader accessibility. Security measures such as password encryption and moderation tools will also be considered to improve user safety and content quality.

REFERENCES

- [E. Gamma, R. Helm, R. Johnson, and J. Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- K. Beck. Test-Driven Development: By Example. Addison-Wesley, 2003.
- J. Smith and A. Doe. "Building Scalable Microblogging Services." Journal of Web Systems, vol. 12, no. 3, pp. 123-145, 2020.